ขบวนผู้กล้า (hero_caravan)

เมื่อออกจากสำนัก อรุณรวบรวมผู้กล้าเป็นขบวน ผู้กล้าแต่ละคนมีรหัสประจำตัวและเกาะเกี่ยวกับคนถัดไปด้วยสายพลัง ผู้กล้าใหม่อาจขอเข้าไปยืนหน้าขบวน ท้ายขบวน หรือต่อท้ายผู้กล้าที่มีรหัสที่กำหนด ขบวนอาจยาวถึงสองแสนคน ผู้ดูแลขบวนจึงต้องรู้เสมอว่า "หัว" และ "ท้าย" อยู่ที่ใด และตอนนี้มีกี่คน โดยไม่ต้องเดินนับใหม่ทุกครั้ง

สิ่งที่ฝึกในด่านนี้: แทรกที่หัว / ท้าย / หลังโหนดที่กำหนด: ใช้ `tail` และ `size` เพื่อให้ทำ `O(1)`



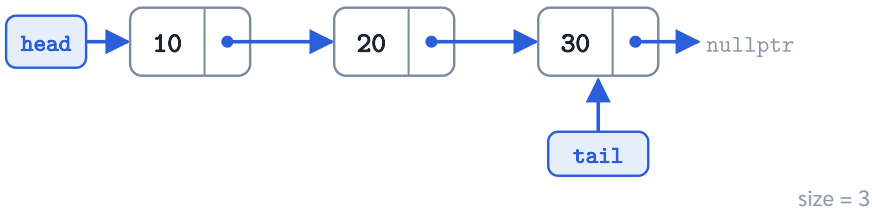
รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันที่ประกาศในไฟล์ `hero_caravan.h` ต่อไปนี้ โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน (โปรแกรมตรวจของผู้จัดทำหน้าที่นั้น)

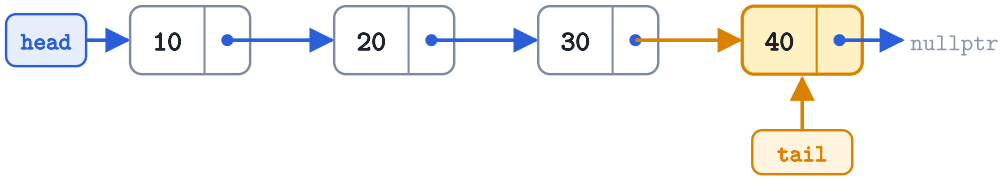
```
struct Node { int data; Node* next; };
struct List {
    Node* head;   // โหนดแรก (nullptr ถ้าว่าง)
    Node* tail;   // โหนดสุดท้าย (nullptr ถ้าว่าง)
    int size;     // จำนวนโหนด
};
void pushFront(List& L, int x);
void pushBack(List& L, int x);
bool insertAfter(List& L, int target, int x);
```

- `pushFront(L, x)`: สร้างโหนดค่า `x` ไว้หน้าสุดของลิสต์ (ลิสต์ว่างก็ใช้ได้)
- `pushBack(L, x)`: สร้างโหนดค่า `x` ไว้ท้ายสุดของลิสต์ (ลิสต์ว่างก็ใช้ได้)
- `insertAfter(L, target, x)`: แทรกโหนดค่า `x` ไว้หลังโหนดแรกสุดที่มี `data == target` แล้วคืน `true`; ถ้าไม่พบ `target` ให้คืน `false` และไม่เปลี่ยนลิสต์
- หลังการเรียกทุกครั้ง `L.size` ต้องเท่ากับจำนวนโหนดจริง `L.head` และ `L.tail` เป็น `nullptr` ทั้งคู่เมื่อลิสต์ว่าง และ `L.tail->next` ต้องเป็น `nullptr`
- เมื่อจบทุกคำสั่ง โปรแกรมตรวจเดินลิสต์ทั้งหมด เทียบกับผลที่ควรได้ และตรวจว่า `L.tail` ชี้โหนดสุดท้ายจริง
- `pushFront` และ `pushBack` ต้องทำงานในเวลา `O(1)` — ห้ามเดินหาท้ายลิสต์ (มี `L.tail` ให้แล้ว) ส่วน `insertAfter` ต้องค้นหา `target` จากหัวลิสต์ ซึ่งใช้เวลา `O(N)` ได้
- ถ้า `insertAfter` แทรกหลังโหนดสุดท้าย ต้องย้าย `L.tail` ไปที่โหนดใหม่ด้วย

ก่อน: ขบวน 10 → 20 → 30

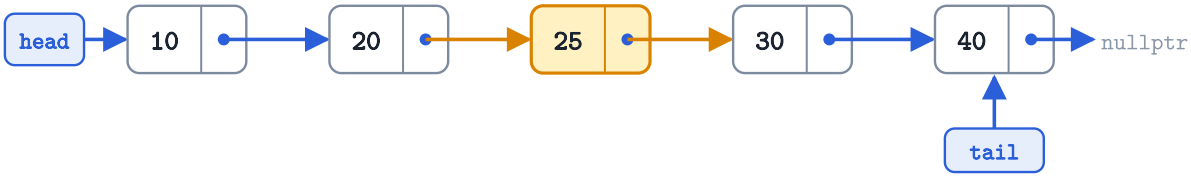


หลัง `pushBack(L, 40)`: ต่อโหนดใหม่ที่ `tail->next` แล้วย้าย `tail`



size = 4

หลัง `insertAfter(L, 20, 25)`: ต่อด้านขวาของโหนดใหม่ก่อน แล้วค่อยต่อด้านซ้าย



size = 5

รูปที่ 1 `List` เก็บ `head` และ `tail` ไว้คู่กัน (โหนดสีเหลืองคือโหนดใหม่ ลูกศรสีส้มคือลูกศรที่เปลี่ยน)

การส่งงาน: ดาวน์โหลดไฟล์แนบ `hero_caravan.cpp` (ไฟล์โครง) และ `hero_caravan.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `hero_caravan.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `hero_caravan.cpp` ห้ามแก้ไขไฟล์ `hero_caravan.h` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp hero_caravan.cpp -o hero_caravan`) ซึ่งตรวจการใช้ `new` / `delete` และความถูกต้องของโหนดให้เหมือนกับระบบจริง

ตัวอย่างการเรียกฟังก์ชัน

การเรียก (เริ่มจากลิสต์ 10 → 20)	ค่าที่คืน	ลิสต์หลังเรียก (หัว → ท้าย)	`tail` ชี้โหนดที่มีค่า
`pushFront(L, 5)`	–	5 → 10 → 20	20
`pushBack(L, 30)`	–	5 → 10 → 20 → 30	30
`insertAfter(L, 20, 25)`	`true`	5 → 10 → 20 → 25 → 30	30
`insertAfter(L, 30, 35)`	`true`	5 → 10 → 20 → 25 → 30 → 35	35
`insertAfter(L, 99, 7)`	`false`	ไม่เปลี่ยน	35

ตารางแสดงการเรียกต่อเนื่องกันตามลำดับบนลิสต์เดียว ขนาด `L.size` หลังการเรียกทั้งห้าครั้งคือ 6

ข้อจำกัด

- $1 \le Q \le 100$
- $0 \le N \le 200\ 000$ และ $0 \le M \le 200\ 000$ ในแต่ละชุด
- ผลรวมของ $N + M$ ทุกชุดในกรณีทดสอบเดียวกันไม่เกิน 400 000 และมีคำสั่ง `A` ไม่เกิน 50 คำสั่งต่อชุด
- $-10^9 \le$ `data`, `target`, $x \le 10^9$ (ค่าซ้ำกันได้ ถ้า `target` มีหลายตัวให้ใช้ตัวแรกจากหัว)
- เวลา 2 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (20 คะแนน) มีเฉพาะคำสั่ง `F`, $N \leq 50$ และ $M \leq 50$
2. (30 คะแนน) $N \leq 1\ 000$ และ $M \leq 1\ 000$
3. (50 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

บรรทัดแรก: จำนวนเต็ม Q คือจำนวนชุดข้อมูล
แต่ละชุด: บรรทัดแรกมี N และ M บรรทัดถัดไปมีค่าเริ่มต้น N ค่า (บรรทัดว่างถ้า $N = 0$) แล้วตามด้วยคำสั่ง M บรรทัด โดยโปรแกรมตรวจสร้าง `List` เริ่มต้นจากค่าเหล่านี้ให้เอง (ถูกต้องทั้ง `head`, `tail`, `size`)
คำสั่งมี 3 แบบ: `F x` เรียก `pushFront(L, x)` · `B x` เรียก `pushBack(L, x)` · `A t x` เรียก `insertAfter(L, t, x)`
พิมพ์ 2 บรรทัดต่อหนึ่งชุด: บรรทัดแรกคือจำนวนคำสั่ง `A` ตามด้วยค่าที่ `insertAfter` คืนตามลำดับ (1 แทน `true` และ 0 แทน `false`) บรรทัดที่สองคือ `L.size` ตามด้วยค่า `data` ของทุกโหนดจากหัวถึงท้ายหลังทำครบทุกคำสั่ง

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
1 2 5 10 20 F 5 B 30 A 20 25 A 30 35 A 99 7	3 1 1 0 6 5 10 20 25 30 35

ตรงกับตารางตัวอย่างการเรียกฟังก์ชัน: คำสั่ง `A` สามครั้งให้ผล 1 1 0 และลิสต์สุดท้ายมี 6 โหนด
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 2, 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
3 0 2 F 4 F 3 1 1 7 F 7 0 0	0 2 3 4 0 2 7 7 0 0

ใช้เฉพาะคำสั่ง `F` ไม่มีคำสั่ง `A` เลย บรรทัดแรกของแต่ละชุดจึงเป็น 0 · ชุดแรกเริ่มจากลิสต์ว่าง จึงต้องตั้ง `tail` ให้ถูกตั้งแต่โหนดแรก
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3